ResearchPilot AI — Architecture Deep Dive

Technical Specification, Multi-Agent Orchestration, Evidence Verification, and Quality Controls

Architecture Paradigm: Hierarchical Supervisor Pattern (Not AI Swarm) | **Language:** Python 3.11+ | **Framework:** Modular Agentic Toolchain
Model Backend: Multi-Tier Gemini 2.5 Flash / Pro | **Gatekeeper:** 9-Point Verification & 12-Check Quality Gate | **Document Version:** 1.0 (Production Blueprint)

Executive Summary & Core Paradigm

ResearchPilot AI is an evidence-grounded research automation architecture designed to produce factual, verifiable, and reproducible research reports from complex inquiries. Unlike conventional generative chatbots that rely on internal parametric memory, ResearchPilot AI decouples statement formulation from factual truth by enforcing a strict **Evidence Ledger**, multi-tier web extraction, adversarial source filtering, claim-level passage verification, and an autonomous adaptive feedback loop.

Why Hierarchical Supervisor Instead of an AI Swarm?

The system intentionally avoids unconstrained peer-to-peer *AI Swarms*. In an AI swarm, independent agents negotiate arbitrarily without deterministic bounds, frequently introducing latency, runaway token expenditure, circular reasoning, and uninspected error propagation. ResearchPilot AI utilizes a **Hierarchical Multi-Stage Orchestrated Workflow (Supervisor Pattern)** governed by the central ResearchOrchestrator. The supervisor maintains state via a thread-safe ResearchStore, executing deterministic validation gates before advancing through pipeline phases.

Core Principle	Architectural Rationale & Enforcement
1. Evidence Grounding Over Memory	A statement is treated as factual only if substantiated by an audited excerpt in the Evidence Ledger. Parametric memory is restricted to syntactic structuring.
2. URL Non-Equivalence to Truth	The presence of a source URL does not validate a claim. The exact text must be retrieved, sanitized, and proven to substantiate the adjacent statement.
3. Negative Boundary Enforcement	Off-topic domains (e.g. EV batteries during a smartphone search) are filtered deterministically at search formulation, source scoring, extraction, and quality gate stages.
4. Rejection of Unsubstantiated Superlatives	Technical specifications ('features 50 MP sensor') cannot prove comparative superiority ('best camera phone'). Superlatives require comparative test evidence.
5. Aspect & Freshness Separation	Daylight photography results cannot substantiate low-light claims. Stale pricing from outdated articles cannot substantiate current market availability.
6. Separation of Concerns	Final user reports provide direct answers with clean links; raw execution logs and rejection reasons are kept isolated in a separate machine-readable Research Audit.

End-to-End Pipeline Phases

Phase	Component	Input & Action	Output & Storage
Phase 1: Ingestion & Planning	QueryAnalyzer ResearchPlanner	Raw user query string. Extracts intent, constraints, primary criteria, and generates 3–6 research subquestions.	QueryIntent & ResearchPlan stored in ResearchStore
Phase 2: Retrieval & Filtering	WebSearchTool SourceFilter PageExtractor	Targeted keyword queries sent to Tavily/DDG. Dual-stage filter applies negative patterns and 0–100 credibility scoring (cut-off: 58). Jina Reader extracts clean Markdown.	Accepted sources registered; rejected sources logged in audit with exact score.
Phase 3: Extraction & Verification	ResearchAnalyst ClaimVerifier GapRecoveryEngine	Scans text, extracts atomic claims + verbatim excerpts. Executes 9-point verification check. Audits question completion criteria.	Evidence ledger items with statuses: SUPPORTED, CONTRADICTED, etc.
Phase 4: Adaptive Recovery	ResearchOrchestrator Feedback Loop	Detects unresolved subquestions. Formulates Round 2 keyword queries with missing criteria and queries web dynamically.	New evidence ledgered; coverage updated to ANSWERED.
Phase 5: Integrity & Quality Gate	CitationValidator ContradictionDetector QualityGate	Validates citation passage and quote fidelity. Analyzes conflicting reviews. 12-point Quality Gate applies bounded auto-repairs.	CitationValidation, ContradictionRecord, QualityGateResult
Phase 6: Synthesis & Presentation	ReportGenerator Streamlit UI (app.py)	Synthesizes direct-answer executive report with comparative table and verified citations. Builds separate JSON audit.	Rendered in UI; exported to Markdown and Audit JSON files.

Detailed Step-by-Step Architecture

Step 1: Ingestion & Query Analysis (QueryAnalyzer)

Source: agent/query_analyzer.py | **Output:** QueryIntent
The query analyzer deconstructs the raw user query into structured semantic components. It avoids generic query expansion and identifies hard boundaries to protect subsequent search stages from semantic drift.

- **Semantic Parsing:** Emits main_intent (e.g. product_recommendation), domain (e.g. smartphones), explicit_constraints (e.g. max_budget: 30000, currency: INR, geo: India).
- **Criteria Isolation:** Distinguishes primary_criteria ('camera quality, low-light, OIS') from secondary_criteria ('battery, display'). Non-requested dimensions (e.g. gaming refresh rate) are explicitly deprioritized.
- **Negative Boundary Extraction:** Generates excluded_topics. For smartphone queries, adds regex terms like ['electric vehicles', 'ev battery', 'charging stations', 'commercial fleet'] to reject false keyword matches.
- **Deterministic Heuristic Fallback:** In case of LLM rate limits (HTTP 429), regular expressions extract currency symbols (Rs, $, INR), numeric ceilings, and geographic tokens.

Step 2: Dynamic Criteria-Driven Research Planning (ResearchPlanner)

Source: agent/planner.py | **Output:** ResearchPlan & List[ResearchQuestion]
Transforms the QueryIntent into 3 to 6 targeted, criteria-driven subquestions. Rather than broad explorations, every question defines a concrete dimension, justification, required evidence, and completion criteria.

Subquestion Schema: question_id (Q1, Q2...) | dimension ('sensor_ois', 'low_light_benchmarks', 'market_pricing')
reason: Why this question must be resolved to answer the user's inquiry.
required_evidence: The specific technical facts or metrics required.
completion_criteria: Objective criteria to mark the question resolved (e.g. 'At least 2 verified devices with validated sensor and OIS details').

Step 3: Targeted Search Formulation & Retrieval (WebSearchTool)

Source: tools/web_search.py & agent/orchestrator.py
Formulates focused search queries rather than conversational phrases:
Search Query Construction: [Topic] + [Constraint] + [Geographic Scope] + [Subquestion Keywords]
Conversational tokens ('what', 'which', 'compare', 'options') are stripped, and query strings are capped at 100 characters. Supports a 3-tier retrieval hierarchy: **Tier 1 (Tavily AI Search API)** for high-signal technical retrieval, **Tier 2 (DuckDuckGo Live Search)** for rate-limit failover, and **Tier 3 (Offline Mock Fallback)** for deterministic test suites. All retrieved URLs are normalized by stripping tracking parameters (utm_source, ref, fbclid).

Step 4: Dual-Stage Strict Source Relevance Filtering (SourceFilter)

Source: tools/source_filter.py | **Threshold:** SCORE >= 58.0 / 100
Prevents SEO spam, irrelevant domains, and low-credibility aggregators from polluting the evidence pool via a dual-stage gatekeeper:

Filtering Stage	Evaluation Mechanism	Outcome & Scoring Impact
Stage 1: Negative Filter	Evaluates title, URL, and snippet against domain negative patterns (e.g. EV battery regex) and user-excluded topics.	Immediate rejection. Score capped at 15.0/100. Logged in audit as 'Off-topic negative filter match'.
Stage 2: Authority Score	Domain credibility evaluation (0–40 pts). .gov/.edu = 40; known tech reviewers (gsmarena, 91mobiles) = 36; social media = 8.	Prevents user-generated unverified social claims from dominating technical research.
Stage 2: Topical Overlap	Calculates proportional overlap with core topic words (0–35 pts) plus primary criteria bonus (+5) and geographic match (+5).	Ensures retrieved page directly addresses user constraints.
Stage 2: Subquestion Fit	Measures specific keyword alignment with the active research subquestion dimension (0–20 pts).	High relevance sources (score >= 58.0) pass to PageExtractor; lower scores logged in audit with exact point tally.

Step 5: Web Content Extraction & Noise Scrubbing (PageExtractor)

Source: tools/page_extractor.py
- **Jina Reader Primary:** Routes URLs through https://r.jina.ai/<url> with Bearer authentication. Cloud headless rendering bypasses client-side JavaScript, ads, and popups.
- **BeautifulSoup4 Fallback:** Direct HTTP GET requests with custom browser headers parse semantic HTML (<article>, <main>) while stripping scripts, styles, navs, and footers.
- **Error & Captcha Rejection:** Evaluates content against regex patterns for *403 Forbidden, Cloudflare captcha, Access Denied*. Fails fast rather than extracting captcha error text as evidence.
- **Noise Sanitization Engine:** Strips Jina metadata headers, cookie banners, navigation breadcrumbs, and transforms complex markdown tables into concise key-value spec strings.

Step 6: Atomic Claim Extraction & Evidence Ledgering (ResearchAnalyst)

Source: agent/analyst.py & storage/research_store.py
Extracts discrete factual assertions and pairs them with exact supporting quotes into the thread-safe **Evidence Ledger**:
- claim_text: Concise factual assertion (<= 250 characters) directly addressing the subquestion.
- supporting_excerpt: Verbatim sentence from source page (<= 200 characters) containing the exact factual substantiation.
- evidence_type: Explicitly labeled as direct (measured spec/test) or inference (reasoned deduction).
- **Deduplication:** Normalizes text strings across multiple sources to avoid redundant claims and registers each under a unique ID (EVD_001).

Step 7: Rigorous 9-Point Claim-Level Verification (ClaimVerifier)

Source: agent/claim_verifier.py
Every claim in the Evidence Ledger must undergo 9 adversarial verification checks before it can be cited in recommendations:

Verification Check	Evaluation Objective	Enforcement Rule & Failure State
1. Exact Passage Support	Does the excerpt substantiate the claim?	Semantic alignment + keyword overlap >= 60%. Failure -> UNVERIFIED.

2. Entity & Variant Match	Does excerpt discuss the exact model/subject?	Named entity match (e.g. Motorola Edge 50 Pro). Failure -> UNVERIFIED.
3. Scope Completeness	Does excerpt substantiate all parts of the claim?	Missing sub-metrics result in PARTIALLY_SUPPORTED.
4. Unmentioned Details	Does claim introduce unmentioned details?	External details absent from excerpt -> flagged UNVERIFIED.
5. Superlatives Check	Does comparative claim have comparative proof?	'Best camera' backed only by spec sheets -> UNVERIFIED.
6. Aspect Alignment	Does excerpt cover the specific aspect claimed?	Low-light claim backed only by daylight review -> UNVERIFIED.
7. Price Freshness	Is current price backed by fresh data?	Stale prices from articles > 1 yr old -> UNVERIFIED.
8. Contradiction Check	Does excerpt refute the claim?	Excerpts containing negation patterns ('failed to') -> CONTRADICTED.
9. Confidence Calibration	Is confidence properly scaled?	Supported facts score >= 0.85; Partial facts score <= 0.70.

Steps 8 & 9: Coverage Auditing & Adaptive Feedback Loop (GapRecoveryEngine)

Sources: agent/gap_recovery.py & agent/orchestrator.py

• **Coverage Evaluation:** Subquestions are classified into ANSWERED (>= 2 supported facts), PARTIALLY_ANSWERED (1 supported fact or partial facts), or UNANSWERED.

• **Autonomous Trigger:** If unresolved gaps remain and search round < max_search_rounds (default: 2), the orchestrator enters the **Adaptive Recovery Loop**.

• **Targeted Follow-up Formulation:** Rather than repeating the initial search, the engine isolates the exact missing criteria:

Round 1 Query: 'camera smartphones under INR 30000 India models'

Round 2 Gap Query: 'camera smartphones India low_light_performance comparative test results night mode'

• Follow-up search results pass through the same strict filtering, extraction, and verification pipelines until coverage criteria are satisfied.

Steps 10 & 11: Citation Validation & Contradiction Detection

Sources: agent/citation_validator.py & agent/contradiction_detector.py

• **Citation Integrity (CitationValidator):** Verifies 5 factors before publication: 1) Source URL exists in storage; 2) Quoted text appears verbatim in source body; 3) Supporting excerpt is present; 4) Named entities match; 5) Factual overlap exists. Emits statuses: VALID, PARTIALLY_VALID, INVALID, or SOURCE_UNAVAILABLE.

• **Contradiction Analysis (ContradictionDetector):** Compares pairwise evidence within each dimension. When opposing assessments appear (e.g. 'sharp night shots' vs 'soft low-light photos'), the detector examines publication dates (detecting firmware changes), testing methodologies (lab chart vs handheld), and variant discrepancies. Generates structured ContradictionRecord objects with resolution_status = 'UNRESOLVED_DISCLOSED' for inclusion in Section 5 of the report.

Step 12: Independent 12-Point Final Quality Gate (QualityGate)

Source: agent/quality_gate.py | **Requirement:** Score >= 0.80 AND Critical Failures == 0

An independent post-generation auditor that inspects the synthesized report against the Evidence Ledger:

12 Quality Gate Audit Checks	Bounded Automated Repair Mechanism
1. Direct Inquiry Answered | 2. Hard Constraints Respected 3. Claims Backed by Ledger | 4. Citations Point to Valid URLs 5. Excerpts Substantiate Claims | 6. Negative Domains Excluded 7. Comparative Claims Grounded | 8. Disagreements Disclosed 9. Stale Details Timestamped | 10. Research Gaps Acknowledged 11. Recommendations Grounded | 12. Standalone Coherence	• **Off-Topic Pruning:** Automatically detects and removes off-topic findings (e.g. commercial vehicle insertions) from report text. • **Superlative Qualification:** Automatically qualifies unbacked comparative claims. • **Gap Injection:** Injects explicit uncertainty disclosures for unresolved questions. • **Decision Rule:** If repairs resolve issues, report is approved (report_passed = True); otherwise rejected.

Steps 13, 14 & 15: Synthesis, Research Audit & UI Presentation

Sources: agent/report_generator.py, models/schemas.py, app.py

• **Report Synthesis (Step 13):** Assembles a user-facing Markdown document structured across 9 sections: Executive Direct Answer, Scope & Methodology, Key Findings & Candidate Comparison Matrix, Evidence Table with clickable links ([Publisher](url)), Trade-offs & Conflicting Evidence, Actionable Recommendations, and Uncertainty Disclosures.

• **Machine-Readable Audit (Step 14):** Emits a complete JSON file (ResearchAudit) detailing query intent, planned subquestions, every query executed, accepted sources, rejected sources with exact scores and reasons, evidence ledger entries, and quality gate scores.

• **UI Presentation (Step 15):** Streamlit frontend renders an Executive Hero Banner, interactive comparison matrices, and a dedicated *Research Audit Tab* featuring metric cards, source rejection inspectors, and one-click JSON/Markdown downloads.

State Transitions & Data Schema Matrix

Pipeline Stage	Component	Output Schema	Store Destination
Step 1: Analysis	QueryAnalyzer	QueryIntent	run.query_intent
Step 2: Planning	ResearchPlanner	ResearchPlan & Questions	run.research_plan
Step 3: Retrieval	WebSearchTool	Raw Search Dicts	In-memory query buffer
Step 4: Filtering	SourceFilter	Accepted vs Rejected	run.sources / rejected_sources
Step 5: Extraction	PageExtractor	Sanitized Markdown	Source.extracted_content
Step 6: Ledgering	ResearchAnalyst	Evidence / EvidenceItem	run.evidence_records
Step 7: Verification	ClaimVerifier	VerifiedClaim (9-point)	Evidence.verification_status
Step 8: Auditing	GapRecoveryEngine	Coverage Status	ResearchQuestion.status
Step 9: Feedback	Orchestrator Loop	SearchRound (Round 2)	run.search_rounds
Step 10: Citations	CitationValidator	CitationValidation	run.citation_validations
Step 11: Conflicts	ContradictionDetector	ContradictionRecord	run.contradictions
Step 12: Quality Gate	QualityGate	QualityGateResult	run.quality_gate_result
Step 13: Synthesis	ReportGenerator	FinalReport (Markdown)	run.final_report
Step 14: Audit Log	ReportGenerator	ResearchAudit (JSON)	run.research_audit
Step 15: UI	Streamlit app.py	Two-Pane Interactive Web UI	Browser Presentation Screen

System Reliability & Verification Guarantee: *ResearchPilot AI is engineered to minimize hallucinations through rigorous relevance gating, claim-level passage alignment, citation verification, contradiction detection, research gap recovery, and post-generation quality gate auditing. While these mechanisms eliminate ungrounded speculative output across standard web research, AI systems operating over open-web corpora cannot claim zero hallucinations. All production deployments preserve explicit uncertainty disclosures and source traceability.*